

KamonBench: A Grammar-Based Dataset for Evaluating Compositional Factor Recovery in Vision-Language Models

Richard Sproat, Stefano Peluchetti
Sakana.ai
{rws,stepelu}@sakana.ai

Abstract

Kamon (家紋, family crests) are an important part of Japanese culture and a natural test case for compositional visual recognition: each crest combines a small number of symbolic choices, but the space of possible descriptions is sparse. We introduce KamonBench, a grammar-based image-to-structure benchmark with 20,000 synthetic composite crests and auxiliary component examples. Each composite crest is paired with a formal kamon description language—*kamon yōgo* (家紋用語)—description, a segmented Japanese analysis, an English translation, and a non-linguistic program code. Because each synthetic crest is generated from known factors, namely container, modifier, and motif, KamonBench supports evaluation beyond caption-level accuracy: direct program-code factor metrics, controlled factor-pair recombination splits, counterfactual motif-sensitivity groups under fixed container-modifier contexts, and linear probes of factor accessibility. We include baseline results for a ViT encoder / Transformer decoder and two VGG n-gram decoders, with and without learned positional masks. KamonBench therefore provides a controlled testbed for sparse compositional visual recognition and factor recovery in vision-language models.

1 Introduction

Kamon (家紋, ‘family crests’) have been a part of Japanese culture since at least the 13th century Kamakura period [25, 5, 3, 17, 26, 18, 19, 27, 23]. The original use was probably as an easily identifiable mark for personal property [27], but soon, samurai began using family crests to distinguish clans during battles. In this usage in particular, kamon were functionally the same as coats of arms in Europe. Traditionally European heraldry has been associated with nobility, and to a large extent that remains true today. For example, in England, if you want to register a coat of arms, you must apply to the *College of Arms* and prove that you are *armigerous*, i.e. have the right to bear arms [6, 7, 22]. In contrast, while kamon were originally associated with noble and warrior families, they have become democratized, so that today almost every family has its own family crest. Kamon are a common sight in Japanese cemeteries, where crests adorn practically every tomb.

Both European heraldry and kamon are associated with *formal languages* that are used to describe the arrangement of motifs within a coat of arms or crest. In British heraldry the formal language is called *blazon* and consists of a rigidly defined set of terms for tinctures (which are part of the graphical vocabulary of heraldry), motifs and their arrangements. For example, Figure 1, left panel, shows a simple coat of arms described by the blazon *azure, a bend or*. Here *azure* means ‘blue’, *bend* denotes the diagonal stripe on the shield, and *or* means ‘gold’—much of the vocabulary of British heraldry deriving from French. The corresponding formal language for kamon is called *kamon yōgō* (家紋用語), which we will henceforth refer to as ‘kamon description language’ (KDL). KDL is less tightly

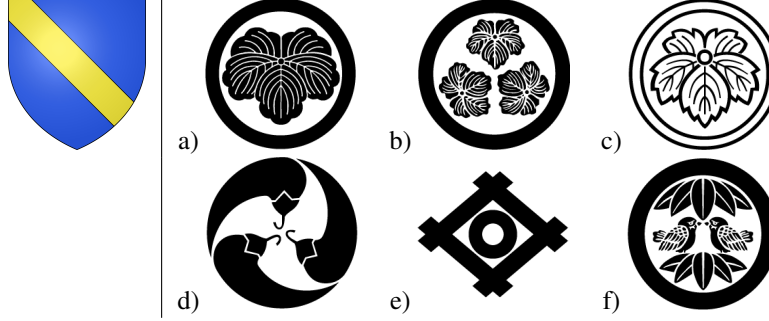


Figure 1: Left: Example of British heraldry, with a simple shield described in blazon as *azure a bend or*. Source: Bear17 (https://commons.wikimedia.org/wiki/File:Azure,_a_bend_or.svg), CC BY-SA 3.0.

Right: Real kamon designs (not from the synthetic KamonBench dataset; shown to illustrate the broader vocabulary that motivates the benchmark): a) circle with ivy (丸に蔦); b) circle with three bottoms-together ivy (丸に尻合わせ三つ蔦); c) circle with demon ivy (総陰丸に鬼蔦); d) chili pepper swirl (唐辛子巴); e) well frame with snake eyes (井桁に蛇の目); f) circle with two rows of five bamboo leaves and facing sparrows (丸に二弾五枚笹に對い雀). Source: [24].

constrained than blazon, but nonetheless is restricted in the ways one can refer to motifs and their arrangements, and the ways in which the descriptions are constructed.

Like European heraldry, kamon have hundreds of motifs, and these motifs can be combined in various ways. In many family crests, one or more motifs are contained within an outer shell such as some type of ring, or a polygonal figure such as a square or hexagon. We call this outer shell a *container*. Figure 1, right panel, shows various examples of complex kamon along with their corresponding KDL.

Two other operation types are important. A *spatial arrangement* changes how copies of a motif are positioned, for example by stacking three copies or placing their heads or bottoms toward the center. A *modification* changes the form or scale of a motif. For example, the ‘demon’ (鬼) modification, which is generally applied to plant motifs, means that the leaf or flower of the plant is depicted with sharpened points [27]. See Figure 1c, and compare the ivy motif in that image with the basic ivy in Figure 1a. Another modification, ‘bean’ (豆), means that the motif is reduced in size (see below, Figure 2b). While there are hundreds of motifs for family crests, the possibilities for spatial arrangements and modifications are limited. Therefore, family crests and their analysis are highly constrained problems. In what follows we use *modifier* as the umbrella term for either a spatial arrangement or a modification. A kamon crest is therefore characterized by three factors: *container*, *modifier*, and *motif*. When an analysis distinguishes the two modifier subtypes, we refer to spatial arrangements and modifications separately.

2 Kamon as a machine learning problem

Kamon are an important part of Japan’s cultural heritage, but what makes them particularly interesting as a machine learning problem, and in particular what makes them interesting as an image-to-structure problem? Like standard image-to-text (or text-to-image) problems, kamon are complex in that ‘scenes’ may consist of multiple elements in various spatial arrangements, and elements may themselves be modified in various ways. But unlike standard image-to-text cases, the modifiers (in the umbrella sense introduced in Section 1) are relatively constrained. This translates, on the language side, to there being a relatively large set of terms corresponding to the set of basic motifs, and an additional set of a few dozen terms to express modifiers of motifs. Containers, motifs that may contain other motifs, are also relatively constrained, again limited to a few dozen cases. But since containment may be recursive, the set of kamon is theoretically unbounded. Even without recursion, the present factor inventory yields approximately 770,000 non-recursive combinations. Thus, kamon comprise a large set of possible images, each related to a string in KDL.

At the same time, kamon represent a *sparse data* scenario, since while there are in principle many examples of image-text combinations found on various sites on the internet, these are largely limited

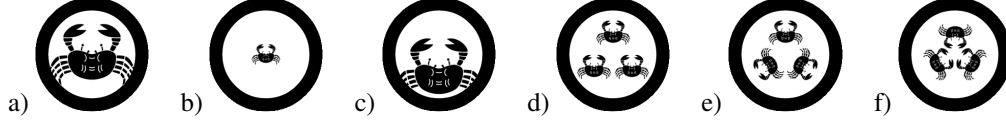


Figure 2: Synthetic examples of crests with various modifiers: a) crab in a circle (丸に蟹); b) *bean* crab in a circle (丸に豆蟹); c) *peeking* crab in a circle (丸に覗き蟹); d) *three stacked* crabs in a circle (丸に三つ盛り蟹); e) *three bottoms-together* crabs in a circle (丸に尻合せ三つ蟹); f) *three heads-together* crabs in a circle (丸に頭合せ三つ蟹).

to the more common crests, typically those associated with particular families. Human experts on kamon need, of course, to be familiar with the various motifs, and understand the modifiers that are possible in the system. A typical manual, such as [27], will list the motifs with a few dozen illustrations of each, and will give a few examples of the various modifiers that are allowed. Humans can fairly easily learn the latter in most cases with just a few examples, or even one: see Section 5.6. For a machine learning system, this same sparsity makes the task a test of whether motifs, containers, and modifiers are represented as reusable visual factors, rather than as memorized whole crest-description pairs.

Our benchmark, described in the next section, consists of synthetic kamon data, generated using a grammar that incorporates a subset of the combination rules of kamon. Why use synthetic data rather than, for example, data from already widely used crests? One reason is that if one is testing LLMs’ abilities at the task of interpreting kamon images into KDL, one would like a dataset that contains examples that the LLM has likely not seen. Since we do not know what web sites proprietary LLMs have used for training, using images similar to those found on various sites is not likely to truly test the LLMs’ knowledge of the domain, since they could well have simply memorized the examples. Synthetic data also gives us access to the underlying generative factors, which makes it possible to ask whether those factors are easily recovered in model outputs and internal representations.

3 The dataset

Kamon motifs are divided into two types, *containers* and other *motifs*. We analyze each composite crest using three factors: an optional container C , a modifier R , and a base motif M . Using simple image manipulation, motifs may receive one of three *spatial arrangements*: 三つ盛り ‘three-stacked’; 尻合せ三つ ‘three bottoms-together’; or 頭合せ三つ ‘three heads-together’. Simple or arranged motifs may also be placed within containers. In that case, two additional *modifications* are relevant: 豆 ‘bean’ (reduced in size); and 覗き ‘peeking’, i.e., out of the bottom of the container. Figure 2 shows some examples of these modifiers using the container 丸 ‘circle’ and the motif 蟹 ‘crab’. Appendix A.2 shows a BNF grammar for the generation process. Note that while the generator supports multiple levels of containment, the released benchmark does not use them: a composite example has either one container or no container.

KamonBench contains 20,000 composite examples. Each composite example consists of a rendered crest, a KDL description, a segmented Japanese analysis of that description, an English translation, and a non-linguistic program label. The Japanese analysis tokenizes the KDL description into Japanese-language parts; the program label records the corresponding generator factors used for factor-aware evaluation. Across the dataset there are 3,513 possible base motifs, 36 possible containers, and six program-label modifier values: null/unmodified, bean, peeking, three-stacked, bottoms-together, and heads-together. In the released composite data, containerless examples use one of the three spatial arrangements; the null/unmodified value occurs for motifs placed directly inside a container.

The dataset also includes auxiliary component examples. A component example is a standalone rendered image of a factor used to generate a composite example: one isolated base motif for every composite example, and one isolated container for every composite example that uses a container. Thus the dataset has 20,000 composite examples, 20,000 base-motif component examples, and 14,116 container component examples, for 54,116 examples in total. The 20,000 composite examples are divided into a 0.8/0.1/0.1 train/development/test split; the corresponding component examples are included in the same split and used for training. They are also available for component-level

checks. We release three recombination splits over the composite examples: (C, M) , (R, M) , and (C, R, M) . The controlled container-motif split uses the same 16,000/2,000/2,000 composite train/development/test sizes as the main split, with 12,918/1,636/1,656 distinct (C, M) groups. Section 5.3 describes these splits and reports results on the controlled container-motif split.

All data can be retrieved at <https://huggingface.co/datasets/SakanaAI/KamonBench>, and the code at <https://github.com/SakanaAI/KamonBench>. The code is released under the MIT License, and the data are released under the CC-BY-NC 4.0 License. The component images bundled with KamonBench (one isolated motif per composite and one container per contained composite) are repackaged in PNG form from the *Rebolforces kamondataset*, a publicly available collection of Japanese kamon motifs originally scraped from a catalogue website that is no longer accessible online (preserved via the Internet Archive); upstream provenance cannot be tracked further. We make no copyright claim over those source images and release KamonBench solely for non-commercial research use.

4 Evaluation enabled by the dataset

Factors and grammar mapping. KamonBench is designed for factor-aware evaluation of image-to-structure prediction. Because each synthetic image is generated from known symbolic choices, each composite example can be represented as

$$Y = (C, R, M),$$

where C is either a single container or null, R records the modifier applied to the motif, and M is the base motif. For contained composites, R may be null/unmodified, a containment modification, or a spatial arrangement. For containerless composites in the released benchmark, R is one of the three spatial arrangements. The released dataset contains no recursive containment, so a single triple (C, R, M) represents every composite example. Appendix A.2 gives the full BNF grammar. The factor metrics below are computed on composite examples; component examples are included in the same splits for training.

Target label spaces. The experiments use three target label spaces over the same images and splits. The Japanese target is a segmented KDL analysis: a Japanese-language sequence that separates the lexical parts of the KDL description and serves as the Japanese sequence target. The English target is a translation sequence. The annotation pipelines for these two target spaces are not released. The *program* target is derived deterministically from the generator components, using non-linguistic codes for the container, modifier, and motif. This makes the (C, R, M) factor metrics direct. For each baseline architecture considered here, we train separate baselines for the Japanese, English, and program targets. This design lets us compare label-space effects while keeping the image distribution fixed.

Aggregate accuracy and edit distance. Aggregate label-space metrics are reported for all three targets on held-out composite examples. Before computing string-level metrics, whitespace is removed and katakana is converted to hiragana. Acc is the fraction of examples whose normalized prediction matches the normalized target. Acc_{NIT} is the same accuracy restricted to composite test examples for which neither the target nor the prediction appears in the training set under the corresponding label-space normalization. CER and TER are both total Levenshtein edit distance divided by total target length, using characters for Japanese and English and program tokens for program-code targets. For composite program codes, contained examples are emitted as three factor tokens, C: . . . , X: . . . , and M: . . . ; containerless examples omit the C: . . . token.

Controlled recombination. Recombination splits hold out combinations of generator factors rather than individual factors. In an (R, M) split, selected modifier-motif pairs are assigned to the test examples. The motif in such a pair still appears in training with other modifiers, and the modifier still appears in training with other motifs. In a (C, M) split, the same construction is applied to container-motif pairs: every test composite contains a container-motif pairing absent from the training composites, while the corresponding container and motif each occur in other training composites. In a (C, R, M) split, the full container-modifier-motif combination is held out. The held-out unit is therefore a combination of familiar factors. Evaluation on these composites asks whether a model can

bind observed visual primitives under new combinations, rather than relying on frequent complete descriptions.

Counterfactual motif sensitivity. The decoder is not constrained by the grammar, so a generated program-code sequence could omit the modifier or motif code, or emit more than one modifier or motif code. We call a prediction parseable when it contains optional container codes, exactly one modifier code, and exactly one motif code. For a fixed container-modifier context $o = (C, R)$ among contained composites, let

$$G_o = \{i : C_i = C, R_i = R\}$$

be the examples with that same container and modifier. Let P be the set of eligible pairs (i, k) drawn from the same G_o with different target motifs, $M_i \neq M_k$, and let p_i indicate that prediction i is parseable. Consider the following three metrics. Motif separation measures sensitivity to motif changes:

$$\text{MotifSeparation} = \frac{1}{|P|} \sum_{(i,k) \in P} \mathbf{1}\{p_i \wedge p_k \wedge \hat{M}_i \neq \hat{M}_k\},$$

which asks whether two examples that differ in target motif also receive different predicted motif codes. A score of 1 means that every eligible pair is separated in the predictions; a score of 0 means that no eligible pair is separated, for example because the model predicts the same motif throughout the container-modifier context. This metric does not require the predicted motifs to be correct.

Pair motif accuracy adds a correctness requirement:

$$\text{PairMotifAcc} = \frac{1}{|P|} \sum_{(i,k) \in P} \mathbf{1}\{p_i \wedge p_k \wedge \hat{M}_i = M_i \wedge \hat{M}_k = M_k\},$$

which requires both predictions in the pair to contain the correct target motif. A score of 1 means that every eligible pair has both motifs correct; a score of 0 means that no eligible pair has both motifs correct. When motif separation is high but pair motif accuracy is low, the model reacts to motif changes but maps them to the wrong motif identities.

Finally, with $\hat{\mathcal{M}}_o = \{\hat{M}_i : i \in G_o, p_i\}$ and $\mathcal{O}$ the set of evaluated container-modifier contexts,

$$\text{CollapsedMotifGroups} = \frac{1}{|\mathcal{O}|} \sum_{o \in \mathcal{O}} \mathbf{1}\{|\hat{\mathcal{M}}_o| \leq 1\}.$$

This group-level score detects whether a fixed container-modifier context is collapsed to a single predicted motif. An individual group contributes 1 when its parseable predictions contain zero or one distinct motif code, and contributes 0 when they contain two or more distinct motif codes. Thus an aggregate score of 1 means that every evaluated container-modifier context is collapsed, while a score of 0 means that none is collapsed.

Linear representation probes. Given a frozen representation $z_i = f_\theta(x_i)$, we train a separate linear probe for each factor $j \in \{C, R, M\}$,

$$q_{\phi,j}(y | z) = \text{softmax}(W_j z + b_j),$$

and evaluate whether the corresponding factor is linearly accessible. For a test slice S , motif cross-entropy is

$$\text{MCE} = -\frac{1}{|S|} \sum_{i \in S} \log q_{\phi,M}(M_i | z_i),$$

computed over examples whose motif label is present in the probe training vocabulary.

KamonBench follows the diagnostic tradition of compositional-generalization benchmarks such as SCAN and CLEVR [13, 12], but casts the problem as visual recognition in a culturally grounded formal language. The benchmark does not claim unsupervised disentanglement or identifiable separated latent dimensions. Instead, it provides supervised factor-recovery and linear-accessibility diagnostics for known generator factors [14]. Appendix A.1 gives further background on compositional generalization, factor recovery, probing, and related work on disentanglement and causal representation learning.

Table 1: Baseline performance on the 2,000 composite test examples. Gray brackets give 95% bootstrap intervals from full test-set resampling with replacement.

Model	Label	CER/TER	Acc	Acc _{NIT}
ViT	Japanese	0.035 [0.030, 0.040]	0.895 [0.882, 0.908]	0.893 [0.878, 0.908]
	English	0.117 [0.109, 0.126]	0.571 [0.548, 0.592]	0.516 [0.492, 0.540]
	Program	0.022 [0.018, 0.026]	0.941 [0.931, 0.951]	0.945 [0.934, 0.955]
VGG masks	Japanese	0.161 [0.152, 0.169]	0.498 [0.476, 0.521]	0.446 [0.421, 0.470]
	English	0.208 [0.198, 0.218]	0.398 [0.377, 0.419]	0.312 [0.289, 0.335]
	Program	0.068 [0.062, 0.075]	0.821 [0.805, 0.836]	0.841 [0.823, 0.858]
VGG no masks	Japanese	0.153 [0.144, 0.162]	0.560 [0.536, 0.581]	0.529 [0.501, 0.554]
	English	0.208 [0.199, 0.218]	0.398 [0.376, 0.419]	0.333 [0.310, 0.355]
	Program	0.101 [0.094, 0.109]	0.732 [0.711, 0.751]	0.783 [0.761, 0.802]

5 Baselines

5.1 Baseline models

We evaluate three baseline families. The first uses a Vision Transformer [4] with an autoregressive Transformer decoder. The second and third use an ImageNet-initialized VGG feature extractor [21, 16] with an n-gram decoder, either with learned position-dependent masks or without masks. The masked variant uses learned image masks so that each output position receives a position-specific view of the crest from outside to inside. Architecture, training, parameter counts, and learned-mask examples are given in Appendix A.3 and Appendix A.6. ViT provides a standard high-capacity image-to-sequence baseline, while the two VGG variants test whether a simpler convolutional encoder can exploit, or do without, an explicit positional bias for reading crests from outer container to inner motif.

5.2 Baseline results

All baselines are trained on the 16,000-composite training split and evaluated on the 2,000-composite test split. We use three target representations: segmented Japanese analysis labels, English translations, and non-linguistic program codes. Table 1 shows that ViT is strongest in all three label spaces, especially on program codes. The two VGG controls are close on English, while no-mask VGG is stronger on Japanese and masked VGG is stronger on program codes.

For program-code outputs, we also map predictions to (C, R, M) factors. We do not apply this decomposition to Japanese or English outputs, where mapping natural-language strings back to generator factors would introduce ambiguity.

Table 2 shows that aggregate program accuracy is mainly a test of motif binding rather than output syntax: all models recover containers and modifiers nearly perfectly, while contained-motif accuracy separates ViT, masked VGG, and no-mask VGG.

Table 3 reports motif sensitivity and pairwise motif accuracy: all models react to motif changes, but pairwise motif accuracy preserves the same ViT, masked VGG, no-mask VGG ranking.

Table 2: Program-label metrics on the test examples. ‘C Acc’ is evaluated only when a container is present; ‘R’ includes spatial arrangements and modifications.

Model	Slice	N	Acc	C Acc	R Acc	M Acc
ViT	All	2000	0.941	1.000	0.994	0.946
	Contained	1431	0.932	1.000	0.992	0.938
	Containerless	569	0.963	N/A	0.998	0.965
VGG masks	All	2000	0.821	0.999	0.989	0.826
	Contained	1431	0.837	0.999	0.984	0.844
	Containerless	569	0.780	N/A	1.000	0.780
VGG no masks	All	2000	0.732	0.999	0.990	0.735
	Contained	1431	0.765	0.999	0.988	0.769
	Containerless	569	0.647	N/A	0.996	0.649

Table 3: Counterfactual motif-sensitivity metrics on the test examples. Each pair shares (C, R) and differs in motif.

Model	Motif sep.	Pair motif acc.	Collapsed motif groups
ViT	1.000	0.929	0.000
VGG masks	1.000	0.870	0.000
VGG no masks	1.000	0.754	0.000

5.3 Controlled container–motif recombination

We evaluate the controlled (C, M) split, which holds out container–motif pairs while retaining primitive-token coverage in training. Because each held-out container and motif appears elsewhere in training, this split targets recombination of familiar factors rather than open-vocabulary recognition. Table 4 reports program-code accuracy and separate container, modifier, and motif accuracies on this controlled (C, M) test split.

On this controlled split, the same ordering holds, with the gap again driven by motif accuracy.

Appendix A.5 reports results on an initial VGG variant featuring a program-label failure that motivated these diagnostics.

5.4 Representation-level factor accessibility

The output diagnostics above ask whether generated descriptions bind the correct factors. We also train linear probes on frozen representations from the program-code baselines to test whether C , R , and M are separately accessible before decoding. For VGG, probes use the concatenation of the per-position VGG features passed to the n-gram decoder. For ViT, probes use the mean-pooled encoder features passed from the ViT image encoder to the Transformer decoder. Probe accuracy is a limited representation test: it shows what a linear readout can recover from the frozen feature, not whether the decoder actually uses that information or whether the information is available nonlinearly.

Table 5 shows that container and modifier labels are almost linearly accessible in all three representations, while motif identity is less accessible in the VGG representations, especially under containment. Table 6 further shows that motif accessibility drops on factor combinations absent from training, with ViT remaining strongest.

We report probe accuracy and motif cross-entropy.

5.5 Few-shot multimodal LLM performance

We provided 20 randomly sampled synthetic examples to two multimodal LLMs, Claude Opus 4.7 Max and GPT 5.4 xhigh. The prompt given to the LLMs can be found in Appendix A.7, and the resulting outputs are shown in Table 14. On these examples, our baseline VGG model made 1 error and the ViT model no errors. In contrast, Claude does not produce a correct

Table 4: Program-label metrics on the (C, M) recombination split.

Model	TER	Acc	C Acc	R Acc	M Acc
ViT	0.035	0.907	0.999	0.994	0.912
VGG masks	0.109	0.711	0.997	0.987	0.716
VGG no masks	0.160	0.572	0.997	0.992	0.572

Table 5: Linear probes on frozen program-model representations. Acc columns report factor probe accuracies; motif accuracy is reported overall, on contained examples, and on containerless examples. M CE is motif cross-entropy.

Model	C Acc	R Acc	M Acc	M Acc _{cont}	M Acc _{noC}	M CE
ViT	1.000	0.988	0.776	0.730	0.893	1.401
VGG masks	0.999	0.987	0.552	0.599	0.434	4.905
VGG no masks	0.999	0.988	0.412	0.430	0.367	6.520

transcription for any of the examples, and GPT produces one correct transcription, although both models sometimes recover individual components such as a container or a motif. This experiment tests a practical confound for kamon evaluation: large proprietary multimodal models may have seen kamon images and KDL descriptions on the web. The results in Table 14 indicate that such exposure, if present, is not sufficient for LLMs to succeed in a few-shot setting on our synthetic benchmark.

5.6 Few-example human and LLM evaluation

We also evaluated whether non-expert humans can learn local aspects of the kamon construction system from a small amount of instruction. Participants first received descriptions of several basic motifs, modifications, and spatial arrangements. They were then presented with ten synthetic examples and asked to identify, for each example, the basic motif, any modification (e.g. 鬼 ‘demon’, 重ね ‘overlapping’), and any spatial arrangement (e.g. 三つ盛り ‘stacked’, 尻合わせ ‘bottoms together’). Participants could select instructions in English (Appendix A.9) or Japanese (Appendix A.10). A sample Google Form question from the questionnaire is shown in Appendix A.11. In addition to questions about the crests, participants were asked to self-report their level of knowledge of kamon, on a 1–4 scale, where 1 is ‘no knowledge’ and 4 is ‘expert’. The task took around 10 minutes for the participants who took part. There were 32 participants. This task is complementary to the KDL transcription prompts: it isolates whether names and operations can be learned from a short tutorial, while the model experiments ask whether those factors can be recombined into structured outputs.

Participants exhibited a wide range of numbers of errors, ranging from no errors for 2 participants, to 10 errors for 1. The largest group of errors related to modification (64%). Most (22) participants reported no knowledge of kamon (1), with the remainder reporting some knowledge (2). There was no significant difference in performance between these two groups. In fact, the 2 participants who made no errors both self-reported as having no knowledge (1), whereas the one participant who made 10 errors self-reported as having some knowledge (2).

We prompted GPT 5.4 xhigh and Claude Opus 4.7 Max with the same ten-example task, using the prompt shown in Appendix A.12. Performance again varied: GPT made 3 errors, whereas Claude made 10 errors. Modification identification was also the hardest category for the two LLMs, accounting for all of GPT’s errors and 60% of Claude’s errors. All human participants performed at least as well as Claude. Relative to GPT, 28% of participants made 2 or fewer errors and therefore outperformed GPT, while 44% made 3 or fewer errors and therefore matched or outperformed GPT. Figure 3 plots the distribution.

Kamon analysis is challenging even for human participants, but the human results show that some aspects of the construction system can be acquired with minimal instruction: 44% of participants matched or exceeded the best LLM performance observed here. If participants’ self-reported prior knowledge is accurate, this performance reflects learning from the provided examples rather than prior formal training in kamon. For proprietary models, by contrast, prior exposure to kamon data cannot be audited.

Table 6: Motif-probe accuracy on test examples whose factor combinations are absent or present in the train split.

Model	Held-out factors	N absent	Acc absent	N present	Acc present
ViT	(C, M)	1450	0.702	550	0.971
	(R, M)	607	0.554	1393	0.873
	(C, R, M)	1698	0.736	302	1.000
VGG masks	(C, M)	1450	0.519	550	0.638
	(R, M)	607	0.091	1393	0.753
	(C, R, M)	1698	0.488	302	0.914
VGG no masks	(C, M)	1450	0.366	550	0.535
	(R, M)	607	0.099	1393	0.548
	(C, R, M)	1698	0.355	302	0.735

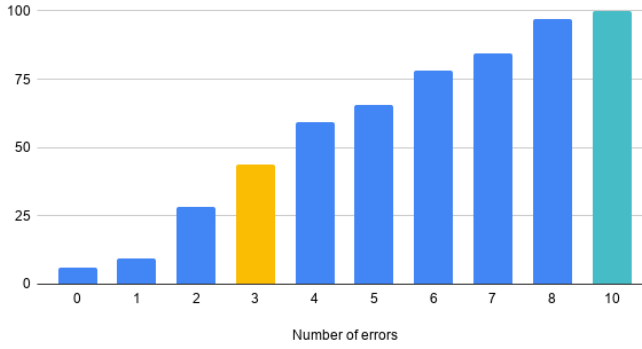


Figure 3: Cumulative error counts for the 32 participants, ranked from the best (2 with no errors) to the worst (1 with 10 errors). Orange bar: GPT 5.4 xhigh; turquoise bar: Claude Opus 4.7 Max. 100% of participants did at least as well as Claude; 44% at least as well as GPT.

6 Conclusion and limitations

KamonBench uses *kamon*, a Japanese heraldic tradition with centuries of historical use and a specialized descriptive vocabulary, as a benchmark for image-to-structure prediction. Its motivation is twofold: *kamon* provide a culturally grounded formal language with meaningful containers, modifiers, and motifs, while the synthetic generator exposes those units as known factors, avoiding likely overlap with web-crawled crest examples and supporting controlled tests of sparse recombination.

The experiments show that this factor-aware view changes what the benchmark measures. Aggregate string metrics identify ViT as the strongest baseline, but program labels reveal that all baselines mostly recover containers and modifiers while motif identity under containment remains the main bottleneck. Controlled (C, M) recombination, counterfactual motif-sensitivity tests, and linear probes separate local factor recognition, compositional binding in outputs, and factor accessibility in frozen representations. The few-example human and multimodal LLM studies further motivate the setting: local aspects of KDL can be learned from limited instruction, while closed proprietary models do not reliably solve the synthetic task from a small prompt.

The benchmark’s control also limits its scope. Generated crests are less polished than professionally rendered *kamon* and differ from real crests in books or on the web. We evaluate a small set of baselines, leave Japanese and English outputs as strings rather than mapping them back into generator factors, and probe only linear accessibility from one feature vector per image. Finally, retraining covers only (C, M) ; the release includes (R, M) and (C, R, M) for future analysis.

References

- [1] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. In *5th International Conference on Learning Representations, Workshop Track Proceedings*, 2017. URL <https://openreview.net/forum?id=HJ4-rAVt1>.
- [2] Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219, 2022. doi: 10.1162/coli_a_00422. URL <https://aclanthology.org/2022.cl-1.7/>.
- [3] Shigeru Chikano. *Nihon Kamon Sōkan* (日本家紋総鑑). Kadokawa Shoten (角川書店), Tokyo, 1993.
- [4] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021. URL <https://arxiv.org/abs/2010.11929>.
- [5] John Dower. *The Elements of Japanese Design*. John Weatherhill, New York, 1971.
- [6] Arthur Charles Fox-Davies. *A Complete Guide to Heraldry*. Dodge Publishing, New York, 1909.
- [7] Stephen Friar and John Ferguson. *Basic Heraldry*. Herbert Press, London, 1993.
- [8] John Hewitt and Christopher D. Manning. A structural probe for finding syntax in word representations. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 4129–4138, Minneapolis, Minnesota, 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1419. URL <https://aclanthology.org/N19-1419/>.
- [9] Irina Higgins, Loic Matthey, Arka Pal, Christopher Burgess, Xavier Glorot, Matthew Botvinick, Shakir Mohamed, and Alexander Lerchner. β -VAE: Learning basic visual concepts with a constrained variational framework. In *International Conference on Learning Representations*, 2017. <https://openreview.net/forum?id=Sy2fzU9gl>.
- [10] Irina Higgins, David Amos, David Pfau, Sebastien Racaniere, Loic Matthey, Danilo Rezende, and Alexander Lerchner. Towards a definition of disentangled representations, 2018. URL <https://arxiv.org/abs/1812.02230>.
- [11] Dieuwke Hupkes, Verna Dankers, Mathijs Mul, and Elia Bruni. Compositionality decomposed: How do neural networks generalise? *Journal of Artificial Intelligence Research*, 67:757–795, 2020. doi: 10.1613/jair.1.11674. URL <https://www.jair.org/index.php/jair/article/view/11674>.
- [12] Justin Johnson, Bharath Hariharan, Laurens van der Maaten, Li Fei-Fei, C. Lawrence Zitnick, and Ross Girshick. CLEVR: A diagnostic dataset for compositional language and elementary visual reasoning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2901–2910, July 2017. URL https://openaccess.thecvf.com/content_cvpr_2017/html/Johnson_CLEVR_A_Diagnostic_CVPR_2017_paper.html.
- [13] Brenden Lake and Marco Baroni. Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In Jennifer Dy and Andreas Krause (eds.), *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pp. 2873–2882. PMLR, 10–15 Jul 2018. URL <https://proceedings.mlr.press/v80/lake18a.html>.
- [14] Francesco Locatello, Stefan Bauer, Mario Lucic, Gunnar Raetsch, Sylvain Gelly, Bernhard Schölkopf, and Olivier Bachem. Challenging common assumptions in the unsupervised learning of disentangled representations. In Kamalika Chaudhuri and Ruslan Salakhutdinov (eds.), *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 4114–4124. PMLR, 09–15 Jun 2019. URL <https://proceedings.mlr.press/v97/locatello19a.html>.

- [15] Francesco Locatello, Michael Tschannen, Stefan Bauer, Gunnar Rätsch, Bernhard Schölkopf, and Olivier Bachem. Disentangling factors of variations using few labels. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=SygagpEKwB>.
- [16] Mayank Mishra, Tanupriya Choudhury, and Tanmay Sarkar. CNN based efficient image classification system for smartphone device. *Electronic Letters on Computer Vision and Image Analysis*, pp. 1–7, 2021.
- [17] Keiichi Morimoto. *Onnamon (女紋)*. Morimoto Dyeing, Kyoto, 2006.
- [18] Yūya Morimoto. *Nihon no Kamon Daijiten (日本の家紋大事典)*. Nihon Jitsugyō Publishers (日本実業出版社), Tokyo, 2013.
- [19] David Phillips. *Japanese Heraldry and Heraldic Flags*. Flag Heritage Foundation, Danvers, MA, 2018.
- [20] Bernhard Schölkopf, Francesco Locatello, Stefan Bauer, Nan Rosemary Ke, Nal Kalchbrenner, Anirudh Goyal, and Yoshua Bengio. Toward causal representation learning. *Proceedings of the IEEE*, 109(5):612–634, 2021. doi: 10.1109/JPROC.2021.3058954. URL <https://doi.org/10.1109/JPROC.2021.3058954>.
- [21] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition, 2015. <https://arxiv.org/abs/1409.1556>.
- [22] Stephen Slater. *The Complete Book of Heraldry*. Lorenz Books, London, 2002.
- [23] Richard Sproat. *Symbols: An Evolutionary History from the Stone Age to the Future*. Springer Nature, Cham, Switzerland, 2023.
- [24] Richard Sproat. 家紋—画像・テキストの新たなチャレンジ (Kamon: Gazō/tekisuto no aratana charenji). In *ANLP*, Utsunomiya, March 2026.
- [25] Hugo Gerard Ströhl. *Japanisches Wappenbuch “Nihon Moncho”*. Verlag von Anton Schroll, Wien, 1906.
- [26] Hitoshi Takasawa. *Kamon no Jiten (家紋の事典)*. Tokyodo Publishers, Tokyo, 2008.
- [27] Hitoshi Takasawa. *Kamon Daijiten (家紋大事典)*. Tokyodo Publishers, Tokyo, 2021.

A Appendix

A.1 Background and related work

KamonBench is designed around three labeled factors of variation per crest: container C , modifier R , and motif M . It provides a suite of factor-aware diagnostics defined in Section 4. This section positions those design choices relative to compositional generalization, factor recovery, linear probing, and related work on disentanglement and causal representation learning.

Compositional generalization. Systematic compositional generalization is usually tested by asking whether a model can recombine primitives in configurations that were rare or absent during training. Text-only benchmarks such as SCAN expose failures to generalize compositionally even when the primitive vocabulary is known [13]. Visual-reasoning benchmarks such as CLEVR use synthetic scenes and executable annotations to separate reasoning skills that are confounded in aggregate question-answering accuracy [12]. KamonBench follows this diagnostic tradition and casts the problem as image-to-structure prediction in a cultural formal language: the input is a rendered crest, and the target is a structured description whose factors are known by construction. In this setting, the relevant primitives are visually grounded motifs, containers, and modifiers; composition requires recognizing these elements and binding them into a valid KDL description. Hupkes et al.’s taxonomy separates several kinds of compositional generalization, including systematicity, productivity, localism, substitutivity, and overgeneralisation [11]. In that taxonomy, our controlled (C, M) holdout is closest to systematicity: each container and motif appears in training, while specific pairs of containers and motifs are withheld, so the test evaluates recombination of known primitives. Hupkes et al.’s substitutivity test concerns synonym substitution, which is not the construction we use; instead, our counterfactual motif groups (Section 4, Table 3) are framed as single-factor interventions on motif identity under a fixed container-modifier context (C, R) .

Factor recovery and probes. In representation learning, disentanglement describes codes in which distinct explanatory factors of variation are separated [9, 10]. This literature has also shown that unsupervised disentanglement is not identifiable without appropriate inductive biases or supervision [14]. KamonBench does not test disentanglement in this stronger sense. Instead, because the generator exposes factor labels by construction, we evaluate supervised factor recovery: whether outputs and frozen representations recover the labeled container, modifier, and motif, and whether those factors remain recoverable under recombination. This framing is consistent with few-label disentanglement work, where limited factor annotations are used for model selection or semi-supervised training [15]. Our representation-level analyses follow the linear-probing tradition for studying what information is accessible in learned representations [1, 8, 2]. The counterfactual factor tests are also aligned with the causal-representation-learning view that useful representations should support controlled interventions on high-level factors [20].

A.2 BNF for synthetic kamon generation

A BNF grammar that defines the possible factor structures is shown in Figure 4.

$\langle kamon \rangle$	$::= \langle contained \rangle$ $\langle spatial-arrangement \rangle \langle MOTIF \rangle$
$\langle contained \rangle$	$::= \langle CONTAINER \rangle \langle complex-motif \rangle$
$\langle complex-motif \rangle$	$::= \langle contained-modifier \rangle \langle MOTIF \rangle$ $\langle contained \rangle$
$\langle contained-modifier \rangle$	$::= \langle modifier \rangle$ $\langle empty \rangle$
$\langle modifier \rangle$	$::= \langle spatial-arrangement \rangle$ $\langle modification \rangle$
$\langle spatial-arrangement \rangle$	$::= \text{三つ盛り} / \text{three-stacked}$ $\text{尻合せ三つ} / \text{three bottoms-together}$ $\text{頭合せ三つ} / \text{three heads-together}$
$\langle modification \rangle$	$::= \text{豆} / \text{bean}$ $\text{覗き} / \text{peeking}$

Figure 4: BNF for kamon generation. Valid $\langle CONTAINER \rangle$ s and $\langle MOTIF \rangle$ s are provided with the released benchmark dataset. The $\langle modifier \rangle$ non-terminal covers both spatial arrangements and modifications. The $\langle empty \rangle$ alternative denotes the null/unmodified value for a motif placed directly inside a container. Containerless composite examples use a spatial arrangement. Note that the recursion on the $\langle complex-motif \rangle$ node, while supported by the generator, is not used in the generation of the current dataset.

A.3 Baseline architecture and training details

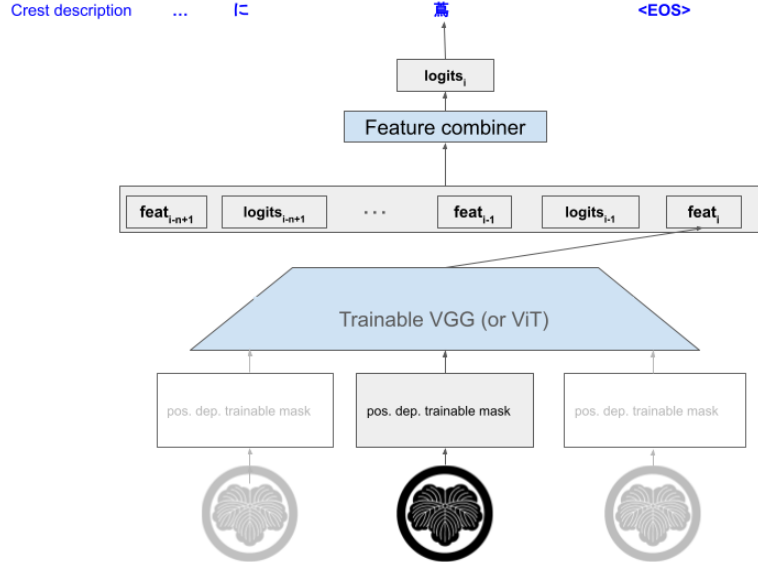


Figure 5: Schematic VGG n-gram decoder family. The blue components are shared across output positions.

In the masked VGG n-gram baseline, I is the input image, P_i is the position-dependent mask, F_i is the feature at position i , L_i is the logit at position i , and FC is the MLP feature combiner that maps

the concatenated context H_i to output logits:

$$\begin{aligned} F_i &= \text{FE}(I \cdot P_i), \\ H_i &= \text{Concat}[F_{i-n+1}, L_{i-n+1}, \dots, F_{i-1}, L_{i-1}, F_i], \\ L_i &= \text{FC}(H_i). \end{aligned}$$

Terms with indices below the first output position are omitted; the n-gram-1 case therefore reduces to $H_i = F_i$.

Tables 7–9 summarize the architecture, optimization, and selected checkpoints for the baselines used in Table 1.

Table 7: Architecture hyperparameters for the three baseline families. All image backbones are initialized from ImageNet-pretrained weights and fine-tuned.

Setting	ViT	VGG masks	VGG no masks
Image encoder	vit_base_patch16_224	VGG16 backbone	VGG16 backbone
Image size	224	224	224
Decoder/interface	4-layer Transformer decoder	MLP n-gram decoder	MLP n-gram decoder
Decoder width	512	512	512
Attention heads	8	N/A	N/A
Decoder context	autoregressive tokens	n-gram length 1	n-gram length 4
Position mechanism	ViT patch features	learned image masks	no learned masks
Dropout	0.1	0.1 in MLP	0.1 in MLP
Token dropout	0.0	N/A	N/A

Table 8: Training and checkpoint-selection settings for the three baseline families.

Setting	ViT	VGG masks	VGG no masks
Optimizer	AdamW	AdamW	AdamW
Main/decoder learning rate	1.5×10^{-4}	10^{-4}	10^{-4}
Backbone learning rate	10^{-5}	same as main LR	same as main LR
Weight decay	10^{-4}	10^{-4}	10^{-4}
Label smoothing	0.02	0.02	0.02
Batch size	64	64	64
Early-stop patience	10	10	10
Selection metric	validation CER/TER	composite validation error	composite validation error

Table 9: Selected checkpoints and parameter counts for the baselines. Parameter counts are computed from checkpoint parameter tensors and exclude non-parameter buffers.

Family	Label	Selected step	Vocabulary size	Max length	Parameters
ViT	English	38,000	2,188	15	105,260,428
ViT	Japanese	78,000	1,858	12	104,920,642
ViT	Program	54,000	3,556	4	106,656,996
VGG masks	English	25,000	2,188	15	138,495,948
VGG masks	Japanese	22,500	1,858	12	138,176,130
VGG masks	Program	23,750	3,556	4	138,645,796
VGG no masks	English	25,000	2,188	15	147,395,532
VGG no masks	Japanese	28,750	1,858	12	146,719,362
VGG no masks	Program	42,500	3,556	4	150,198,564

The reported experiments fit on a single H100 GPU node.

A.4 Reduced-training-data baseline sweep

To measure data scaling under the same baselines, we retrained ViT, masked VGG, and no-mask VGG on deterministic training subsets containing 2,500, 5,000, or 10,000 composite examples. We evaluate on the standard development and test splits.

Table 10 reports the same aggregate composite-test metrics as Table 1. Table 11 reports the same program-code factor metrics as Table 2.

Table 10: Reduced-training-data performance on the standard test data. Train size counts selected training examples. Gray brackets give 95% bootstrap intervals.

Train	Model	Label	CER/TER	Acc	Acc _{NIT}
2,500	ViT	Japanese	0.273 [0.262, 0.285]	0.368 [0.347, 0.390]	0.387 [0.364, 0.409]
		English	0.303 [0.293, 0.315]	0.234 [0.216, 0.249]	0.235 [0.216, 0.252]
		Program	0.219 [0.211, 0.227]	0.424 [0.402, 0.445]	0.477 [0.453, 0.501]
	VGG masks	Japanese	0.387 [0.378, 0.396]	0.099 [0.086, 0.112]	0.082 [0.069, 0.096]
		English	0.413 [0.403, 0.422]	0.085 [0.072, 0.097]	0.067 [0.055, 0.078]
		Program	0.284 [0.275, 0.293]	0.276 [0.256, 0.295]	0.370 [0.343, 0.396]
	VGG no masks	Japanese	0.411 [0.401, 0.420]	0.103 [0.089, 0.117]	0.097 [0.082, 0.110]
		English	0.408 [0.399, 0.419]	0.102 [0.089, 0.116]	0.093 [0.079, 0.107]
		Program	0.337 [0.329, 0.344]	0.146 [0.132, 0.162]	0.320 [0.289, 0.352]
5,000	ViT	Japanese	0.157 [0.147, 0.167]	0.600 [0.580, 0.621]	0.614 [0.592, 0.635]
		English	0.221 [0.211, 0.231]	0.359 [0.339, 0.380]	0.351 [0.330, 0.373]
		Program	0.117 [0.109, 0.124]	0.691 [0.672, 0.710]	0.720 [0.700, 0.740]
	VGG masks	Japanese	0.319 [0.310, 0.329]	0.203 [0.185, 0.222]	0.176 [0.158, 0.194]
		English	0.334 [0.325, 0.344]	0.170 [0.153, 0.186]	0.136 [0.120, 0.151]
		Program	0.199 [0.190, 0.209]	0.479 [0.456, 0.500]	0.574 [0.547, 0.597]
	VGG no masks	Japanese	0.306 [0.295, 0.316]	0.251 [0.231, 0.270]	0.243 [0.222, 0.264]
		English	0.326 [0.316, 0.337]	0.195 [0.178, 0.213]	0.173 [0.155, 0.190]
		Program	0.277 [0.268, 0.285]	0.279 [0.259, 0.297]	0.452 [0.421, 0.483]
10,000	ViT	Japanese	0.070 [0.062, 0.077]	0.805 [0.788, 0.822]	0.810 [0.790, 0.827]
		English	0.142 [0.133, 0.152]	0.515 [0.493, 0.535]	0.485 [0.462, 0.507]
		Program	0.047 [0.042, 0.052]	0.876 [0.862, 0.889]	0.889 [0.874, 0.902]
	VGG masks	Japanese	0.215 [0.205, 0.225]	0.393 [0.371, 0.414]	0.354 [0.329, 0.376]
		English	0.245 [0.235, 0.255]	0.324 [0.301, 0.342]	0.266 [0.244, 0.285]
		Program	0.107 [0.100, 0.115]	0.719 [0.698, 0.738]	0.770 [0.750, 0.790]
	VGG no masks	Japanese	0.199 [0.189, 0.209]	0.477 [0.454, 0.498]	0.463 [0.438, 0.487]
		English	0.240 [0.230, 0.250]	0.348 [0.328, 0.370]	0.308 [0.286, 0.330]
		Program	0.173 [0.164, 0.182]	0.547 [0.524, 0.570]	0.641 [0.615, 0.667]

Table 11: Reduced-training-data metrics on the standard test data, matching Table 2.

Train	Model	Slice	N	Acc	C Acc	R Acc	M Acc
2,500	ViT	All	2000	0.424	0.992	0.965	0.436
		Contained	1431	0.416	0.992	0.955	0.431
		Containerless	569	0.445	N/A	0.988	0.448
	VGG masks	All	2000	0.276	0.997	0.950	0.280
		Contained	1431	0.336	0.997	0.945	0.340
		Containerless	569	0.123	N/A	0.963	0.128
	VGG no masks	All	2000	0.146	0.990	0.935	0.154
		Contained	1431	0.164	0.990	0.934	0.169
		Containerless	569	0.102	N/A	0.938	0.116
5,000	ViT	All	2000	0.691	0.999	0.986	0.696
		Contained	1431	0.684	0.999	0.981	0.691
		Containerless	569	0.707	N/A	0.998	0.707
	VGG masks	All	2000	0.479	0.997	0.972	0.485
		Contained	1431	0.564	0.997	0.966	0.571
		Containerless	569	0.265	N/A	0.988	0.271
	VGG no masks	All	2000	0.279	0.997	0.966	0.282
		Contained	1431	0.316	0.997	0.960	0.317
		Containerless	569	0.185	N/A	0.979	0.195
10,000	ViT	All	2000	0.876	0.999	0.992	0.880
		Contained	1431	0.868	0.999	0.990	0.874
		Containerless	569	0.896	N/A	0.998	0.896
	VGG masks	All	2000	0.719	1.000	0.986	0.722
		Contained	1431	0.770	1.000	0.982	0.774
		Containerless	569	0.589	N/A	0.996	0.591
	VGG no masks	All	2000	0.547	0.998	0.978	0.552
		Contained	1431	0.598	0.998	0.971	0.602
		Containerless	569	0.420	N/A	0.993	0.427

A.5 Initial VGG program-label failure

An initial masked VGG n-gram-2 program baseline, trained without label smoothing or weight decay, exposed a failure mode that aggregate string metrics would otherwise compress into a single low score. At the time, this motivated checking similar VGG models that do not exhibit the same collapse.

Table 12 compares this initial run with masked and no-mask VGG variants and ViT. The initial masked n-gram-2 model learns the program schema and recovers the container and modifier but obtains 0.000 contained accuracy and 0.000 contained motif accuracy. The closest tested masked configuration that removed the collapse was the regularized n-gram-1 model selected on composite validation error. Removing masks under the same regularized composite-selection setup also removed the collapse: no-mask n-gram-2 reached 0.737 contained motif accuracy. The no-mask baseline uses a wider n-gram-4 context.

Table 12: VGG n-gram-2 program-label failure and comparison with other models.

Model and setting	TER	Acc	Contained Acc	Contained M
VGG, n-gram-2 masks	0.317	0.151	0.000	0.000
VGG, n-gram-2 no masks + reg.	0.106	0.722	0.731	0.737
VGG, n-gram-1 masks + reg.	0.068	0.821	0.837	0.844
VGG, n-gram-4 no masks + reg.	0.101	0.732	0.765	0.769
ViT	0.022	0.941	0.932	0.938

Table 13 reports the motif prediction concentration behind the initial VGG failure. The contained target motifs are highly dispersed, but the initial n-gram-2 VGG program model maps the entire contained slice to one motif token. This collapsed token, M:0808, appears only 13 times in the training split and never appears as a target motif in the development or test splits.

Table 13: Motif prediction concentration in the initial n-gram-2 VGG collapsed program-label outputs. ‘Top target’ and ‘Top pred.’ give the most common target and predicted motif tokens in each slice, with counts in parentheses.

Model	Slice	N	M Acc	Top target	Top pred.	Top wrong pred.
Initial VGG n2	All	2000	N/A	M:00328 (5)	M:0808 (1431)	M:0808 (1431)
	Contained	1431	0.000	M:24407 (4)	M:0808 (1431)	M:0808 (1431)
	Containerless	569	0.529	M:26611 (3)	M:1053 (4)	M:1053 (4)
ViT	All	2000	0.946	M:00328 (5)	M:00328 (5)	M:26704 (3)
	Contained	1431	0.938	M:24407 (4)	M:24407 (4)	M:26704 (3)
	Containerless	569	0.965	M:26611 (3)	M:26611 (3)	M:0748 (2)

A.6 Learned masks for masked VGG baselines

Figure 6 shows learned position-dependent masks for the masked VGG baselines.

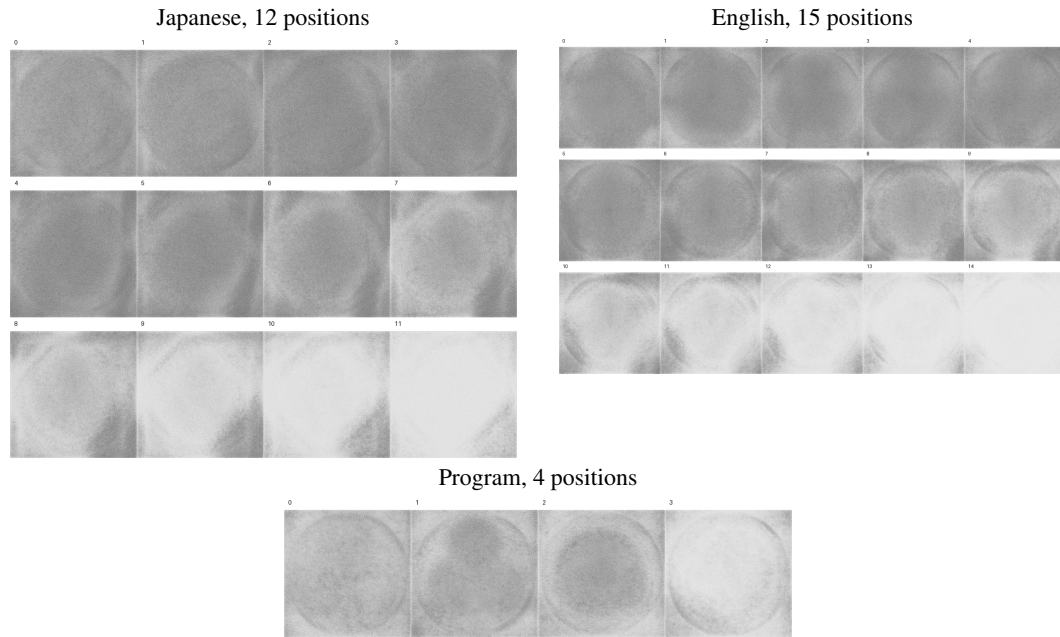


Figure 6: Positional masks from the masked VGG baselines. Images are inverted: darker regions indicate larger retained mask values.

A.7 LLM prompt for 20 random examples

The following shows the prompt given to the LLMs to produce the outputs shown in Table 14.

Japanese kamon (family crests, 家紋) have a description language called 家紋用語 that is similar to blazon in European heraldry in that it defines the elements of the crest and the spatial relationships between them.

The following are some examples of kamon and their associated descriptions:



丸に 桐



丸に 尻合わせ 三つ 桐



総陰 丸に 鬼 桐



唐 辛子 巴



井 桁に 蛇の 目

Based on these examples, provide descriptions for the following numbered kamon:



11. 
12. 
13. 
14. 
15. 
16. 
17. 
18. 
19. 
20. 

Respond with the description, with numbers matching those of the images above, one per line as follows:

1. (description 1)
2. (description 2)
3. (description 3)
(etc.)

A.8 Few-shot multimodal LLM performance

Table 14 shows the 20 sampled synthetic examples used for the Japanese LLM prompt, with VGG and ViT outputs where the sampled image is present in the test predictions, and two large language models, Claude Opus 4.7 Max and GPT 5.4 xhigh. The prompt given to the LLMs is in the previous section, Appendix A.7.

The cases where the predicted string from the model matches the target transcription are shown with a check mark; a dash means that the sampled image is not part of the test-prediction JSONL files. Claude does not produce a correct transcription for any of the examples, and GPT produces one correct transcription, although both models sometimes recover individual components such as a container or a motif.

Table 14: LLM performance on synthetic Japanese data. For comparison, the baseline VGG and ViT results are shown, with check marks if the prediction matched the target. LLM details: Claude Opus 4.7 Max, GPT 5.4 xhigh.

Image	Description/Translation	VGG	ViT	Claude	GPT
	隅立て角に秋津洲 浜/Akitsushima hama in a corner-standing corner	✓	✓	隅立て井桁に花菱	唐花菱
	雪輪に三つ割り南天/three- split nanten in a snow ring	✓	✓	雪輪に頭合せ三つ茗 荷	木瓜に三つ南天
	丸に左稲の丸/circle of left rice plant in a circle	✓	✓	丸に稲	丸に抱き稲
	細隅入り角に変わり切り竹 笹/modified cut bamboo grass in a thin corner-entering square	✓	✓	雪輪菱に笹	竹菱
	糸輪に台梅鉢/stand plum blos- som crest in a thread ring	✓	✓	丸に梅鉢	丸に五瓜に唐花
	瓜輪に結び柏/tied-together oak leaves in a melon ring	✓	✓	丸に三つ輪違い	三つ葵巴
	源氏輪に違い一重亀 甲/alternating single-layered tortoiseshell pattern in a Genji ring	✓	✓	丸に輪宝	源氏車に輪違い
	雪輪に江原柏/Ebara oak leaves in a snow ring	✓	✓	雪輪に矢	木瓜に抱き矢
	竹輪に中陰裏唐花/medium- shaded reverse karakusa in a bam- boo ring	✓	✓	雪輪に花菱	丸に木瓜に唐花
	藤輪に二枚葉蔓梔の葉/two- leaves vine kaji leaves in a wister- ia ring	✓	✓	丸に抱き茗荷	丸に抱き柏
	尻合せ三つ大割鬼蔦/bottom- joined three large split demon ivy	✓	✓	三つ盛り柏	尻合せ三つ葵
	三つ盛り外三つ割り蔦/three- piled outer-three-split ivy	✓	✓	頭合せ三つ柏	尻合せ三つ蔦
	二重輪に隅立て四つ目/corner- standing four-eyed in a double ring	✓	✓	丸に武田菱	丸に四つ目菱
	三つ盛り南天蝶/three piled nan- ten butterflies	✓	✓	三つ盛り蔦	三つ盛り茗荷
	三つ盛り千鳥/three piled chi- dori	尻合せ三つ千鳥	✓	三つ盛り雀	✓
	三つ盛り三階菱橘/three- piled three-tiered water caltrop tachibana	✓	✓	三つ盛り割菱	三つ盛り三階菱
	平角に木瓜形/quince shape in a flat square	✓	✓	石持ち地抜き木瓜	角に木瓜
	八角に抱き芭蕉/embracing ba- nana plant in an octagon	✓	✓	八角に抱き笹	八角に違い鷹の羽
	雪輪に寄り懸り輪違 い/alternating leaning-hanging rings in a snow ring	✓	✓	雪輪に輪違い	木瓜に輪違い
	頭合せ三つ休花菱/head- joined three Rikyū flower dia- monds	✓	✓	三つ盛り花菱	尻合せ三つ桔梗

A.9 Human instructions: English

Japanese kamon (family crests, 家紋) have a description language called 家紋用語 that is similar to blazon in European heraldry in that it defines the elements of the crest and the spatial relationships between them, along with other modifications.

For example some modifications of motifs are as follows:

Opposite:

Patterns with a specific orientation are placed facing each other from above and below, or from the left and right.

E.g.:

opposite



mountain
shape



opposite mountain
shapes

Demon:

With plant motifs this depicts leaves and petals with sharply pointed tips.

E.g.:

demon



chrysanthemum
diamond



demon
chrysanthemum
diamond

Overlapping:

This refers to a design where two or more crests are overlapped using parts of each crest.

E.g.:

Overlapping



three
cedars



overlapping three
cedars

Hugging:

The upper and lower parts are drawn close together, or their lower parts are crossed, to give the impression of them closely embracing. Often, they are drawn in a circular shape.

E.g.:

Hugging



paper
mulberry leaf



hugging paper
mulberry leaf

Apart from modifications, another feature is that simple motifs may be combined in various ways, and these ways are highly constrained and programmatic. For example, a simple motif may be stacked in groups of 3, arranged in a circle of 3 with their heads together or arranged in a circle with their bottoms together. Note that what counts as "head" and "bottom"

depends on whatever is the normal presentation direction for the motif, with head being upwards, and bottom downwards.

For example, the image below shows the normal form of the clam motif:



clam

So, given the above description, we get the following combinations:



three stacked clams



three heads-together clams

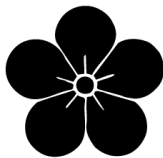


three bottoms-together clams

Consider now the following basic unmodified motifs:



paper mulberry leaf



plum



orange

Based on these motifs in their normal orientation, and the explanations provided above, in the questionnaire, select the correct **motif**, **modification (or none)**, and **arrangement** for the 10 example combinations in the [QUESTIONNAIRE](#).

A.10 Human instructions: Japanese

日本の家紋には、ヨーロッパの紋章学における「紋章記述 (blazon)」に似た「家紋用語」と呼ばれる言語的な慣習があり、この家紋用語を用いることで、家紋の構成要素とそれらの空間的な関係を定義することができます。

例えば、モチーフのいくつかの改造の例は以下のとおりです。

違い:

特定の向きを持つ紋形を上下、あるいは左右から向かい合わせる。

例:

違い



山形



違い山形

鬼:

植物紋で葉や花卉の先端を鋭く尖らせて描くものである。

例:

鬼



菊菱



鬼菊菱

重ね:

2つ以上の紋を、紋の一部で重ね合わせたものをいう。

例:

重ね



三本杉



重ね三本杉

抱き:

上下部分を近づけ、あるいは下部を交差させて密接に抱き合っているように描く。多くの場合円形に描かれる。

例:

抱き



梶の葉



抱き梶の葉

改造以外、他の家紋の特徴は、単純なモチーフ（何らかの図柄）を様々な方法で組み合わせることができる点です。ただし、これらの組み合わせ方は厳密に制約され、体系化されています。例えば、単純なモチーフを、三つ積み重ねる（三つ盛り）、上部を揃えて円形に配置する（頭合せ三つ）、または下部を揃えて円形に配置する（尻合せ三つ）、といった組み合わせが挙げられます。ここで、「頭」と「尻」は、モチーフの通常の表示方向によって決まり、頭は上、尻は下となります。

例えば、以下のイメージは、「蛤」のモチーフの通常の形を示しています。



蛤

上記の記述に基づくと、その右の三つの画像に示すような組み合わせが得られます。



三つ盛り蛤



頭合せ三つ蛤

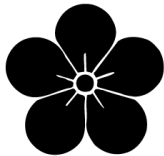


尻合せ三つ蛤

次に、以下の基本的な未改造のモチーフについて考えてみましょう。



梶の葉



梅



橘

上記の説明と、通常の向きにおけるこれらのモチーフに基づいて、アンケートでは、提示された例の組み合わせに対して、[アンケート](#)にある10個の例の正しいモチーフ、改造(またはなし)、および組み合わせを選択してください。

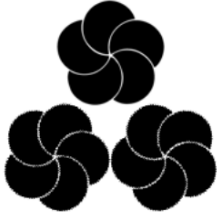
A.11 Questionnaire sample

家紋アンケート/Kamon Questionnaire

* Indicates required question

例/Example 1

A *



頭合せ三つ/3 heads-together 三つ盛り/3 stacked 尻合せ三つ/3 bottoms-together

組み合わせ/Arrangement	☐	☐	☐
-------------------	-----------------------	-----------------------	-----------------------

B *

違い/opposite 鬼/demon 重ね/overlapping 抱き/hugging なし/None

改造/Modification	☐	☐	☐	☐	☐
-----------------	-----------------------	-----------------------	-----------------------	-----------------------	-----------------------

C *

梶の葉/paper mulberry leaf 橘/orange 梅/plum

モチーフ/ motif	☐	☐	☐
-------------	-----------------------	-----------------------	-----------------------

[Back](#) [Next](#) [Clear form](#)

A.12 LLM prompt for 10 kamon rated by humans

Japanese kamon (family crests, 家紋) have a description language called 家紋用語 that is similar to blazon in European heraldry in that it defines the elements of the crest and the spatial relationships between them, along with other modifications.

For example some modifications of motifs are as follows:

違い (opposite):

特定の向きを持つ紋形を上下、あるいは左右から向かい合わせる。

Patterns with a specific orientation are placed facing each other from above and below, or from the left and right.

E.g.:

違い
(opposite)



山形
(mountain
shape)



違い山形 (opposite
mountain shape)

鬼 (demon):

植物紋で葉や花弁の先端を鋭く尖らせて描くものである。

With plant motifs this depicts leaves and petals with sharply pointed tips.

E.g.:

鬼
(demon)



菊菱
(chrysanthemum
diamond)



鬼菊菱 (demon
chrysanthemum
diamond)

重ね (overlapping):

2つ以上の紋を、紋の一部で重ね合わせたものをいう。

This refers to a design where two or more crests are overlapped using parts of each crest.

(.)

E.g.:

重ね
(overlapping)



三本杉
(three
cedars)



重ね三本杉
(overlapping three
cedars)

抱き (hugging):

上下部分を近づけ、あるいは下部を交差させて密接に抱き合っているように描く。多くの場合円形に描かれる。

The upper and lower parts are drawn close together, or their lower parts are crossed, to give the impression of them closely embracing. Often, they are drawn in a circular shape.

E.g.:

抱き
(hugging)



梶の葉 (paper
mulberry leaf)



抱き梶の葉 (hugging
paper mulberry
leaf)

Apart from modifications, another feature is that simple motifs may be combined in various ways, and these ways are highly constrained and programmatic. For example, a simple motif may be stacked in groups of 3 (三つ盛り), arranged in a circle of 3 with their heads together (頭合せ三つ) or arranged in a circle of 3 with their bottoms together (尻合せ三つ). Note that what counts as "head" and "bottom" depends on whatever is the normal presentation direction for the motif, with head being upwards, and bottom downwards.

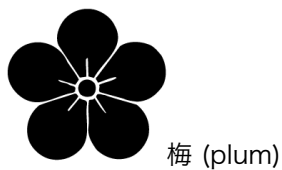
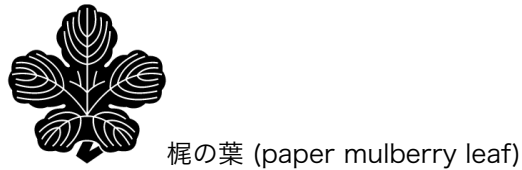
For example, the image below shows the normal form of the clam (蛤) motif:



So, given the above description, we get the following combinations:



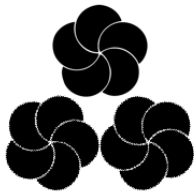
Consider now the following basic unmodified motifs:





橘 (orange)

Based on these motifs in their normal orientation, and the explanations provided above, pick the correct motif, modification and orientation for the following example combinations.



1.



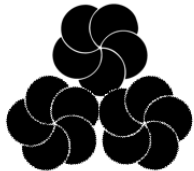
2.



3.



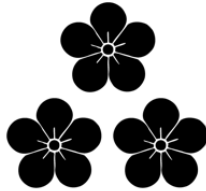
4.



5.



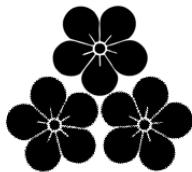
6.



7.



8.



9.



10.

Respond with the description, with numbers matching those of the images above, one per line as follows:

1. (description 1)
2. (description 2)
3. (description 3)

(etc.)